Sentinel

Technical Manual

Concurrent Sensor Monitoring System

Pedro Medina

github.com/pedromedinatech/sentinel

Version	1.0
Date	April 14, 2026
Language	Python 3.14

Contents

1 Introduction **3**
- 1.1 Overview . . . 3
- 1.2 Motivation . . . 3
- 1.3 Scope . . . 3
- 1.4 Technology Stack . . . 3

2 System Architecture **4**
- 2.1 High-Level Overview . . . 4
- 2.2 Components . . . 5
  - 2.2.1 Server . . . 5
  - 2.2.2 Client . . . 5
  - 2.2.3 Monitor . . . 5
  - 2.2.4 Shared . . . 5
- 2.3 Concurrency Model . . . 5
- 2.4 Design Decisions . . . 7
  - 2.4.1 Why asyncio over threads? . . . 7
  - 2.4.2 Why separate PHYSICAL_LIMITS from ALERT_THRESHOLDS? 7
  - 2.4.3 Why hash the shared secret? . . . 7

3 Communication Protocol **8**
- 3.1 Overview . . . 8
- 3.2 Message Types . . . 8
- 3.3 Message Schemas . . . 8
  - 3.3.1 register . . . 8
  - 3.3.2 reading . . . 9
  - 3.3.3 ack . . . 9
  - 3.3.4 rejected . . . 9
- 3.4 Authentication Flow . . . 9
- 3.5 Reading Flow . . . 11

4 Processing Pipeline **12**
- 4.1 Overview . . . 12
- 4.2 Stage 1: Validator . . . 14
- 4.3 Stage 2: ThresholdDetector . . . 14
- 4.4 Stage 3: ZScoreDetector . . . 14
- 4.5 Relationship to False Data Injection . . . 15

5 Design Patterns **16**
- 5.1 Overview . . . 16
- 5.2 Observer . . . 16
- 5.3 Strategy . . . 16
- 5.4 Template Method . . . 17
- 5.5 Singleton . . . 18

6 UML Diagrams **19**
- 6.1 Class Diagram . . . 19
- 6.2 Authentication Sequence Diagram . . . 20

6.3 Reading Processing Sequence Diagram . . . 21
6.4 Processing Pipeline Activity Diagram . . . 22
6.5 Client Connection Lifecycle . . . 24

7 Configuration Reference 25
7.1 Network . . . 25
7.2 Authentication . . . 25
7.3 Pipeline . . . 25
7.4 Physical Limits . . . 26
7.5 Alert Thresholds . . . 26

8 Setup and Usage 27
8.1 Requirements . . . 27
8.2 Installation . . . 27
8.3 Running the System . . . 27
8.3.1 Start the server . . . 27
8.3.2 Connect a real sensor client . . . 27
8.3.3 Connect a simulated sensor client . . . 28
8.3.4 Inject anomalous values . . . 28
8.3.5 Available sensor types . . . 28
8.4 Demo Scenario . . . 28
8.5 Stopping the System . . . 29

9 Conclusion 30
9.1 Current Limitations . . . 30
9.2 Possible Extensions . . . 30
9.3 Final Remarks . . . 31

1 Introduction

1.1 Overview

Sentinel is a concurrent sensor monitoring system implemented in Python 3.14. It provides a server capable of handling multiple simultaneous sensor clients, processing incoming readings through a multi-stage validation pipeline, and notifying subscribers in real time when anomalous values are detected.

The system is designed around three core principles: separation of concerns, loose coupling between components, and explicit communication contracts. Each of these principles is reflected in the architecture and in the design decisions described throughout this document.

1.2 Motivation

Modern monitoring systems must handle data from multiple sources simultaneously without blocking, detect anomalies reliably without generating excessive false positives, and notify the appropriate parties without tight coupling between the detection logic and the notification logic.

Sentinel addresses these requirements through a combination of asynchronous concurrency, a layered processing pipeline, and an event-driven notification mechanism based on the Observer pattern.

1.3 Scope

This document covers the internal architecture of Sentinel, the communication protocol between clients and the server, the processing pipeline design, the design patterns applied, and the configuration reference. It is intended for developers who want to understand, extend or evaluate the system.

1.4 Technology Stack

Component	Technology
Language	Python 3.14
Concurrency	asyncio
System metrics	psutil 7.2.2
Transport protocol	TCP sockets
Message format	JSON
Authentication	Shared secret with SHA-256 hashing
Statistical method	Z-score with sliding window

Table 1: Technology stack

2 System Architecture

2.1 High-Level Overview

Sentinel follows a layered architecture with four distinct components that communicate through well-defined interfaces. The server acts as the central hub — it accepts incoming sensor connections, processes readings through a pipeline, and delegates notifications to registered subscribers via an event bus.

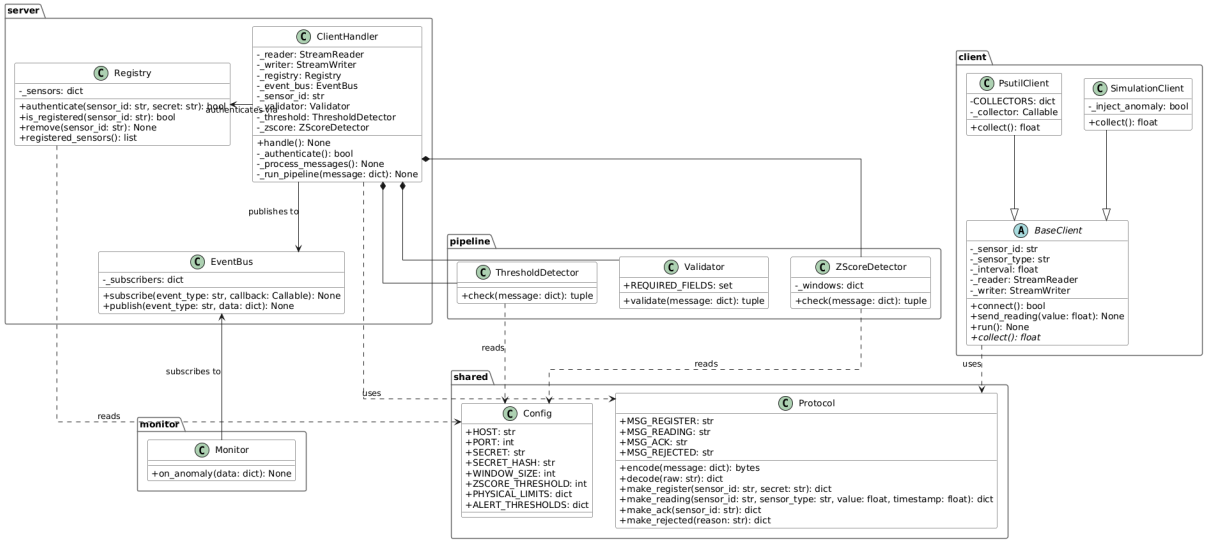

Figure 1: Class diagram

2.2 Components

2.2.1 Server

The server is the core of the system. It manages concurrent client connections using Python's `asyncio` event loop where each client runs in its own coroutine, allowing the server to handle multiple sensors simultaneously without spawning threads. It owns two shared instances that persist for the duration of the session: the `Registry` and the `EventBus`.

2.2.2 Client

Sensor clients connect to the server over TCP, authenticate with a shared secret, and send periodic readings. Two client types are provided: `PsutilClient`, which collects real system metrics (CPU, RAM, disk, network) using the `psutil` library, and `SimulationClient`, which generates configurable readings including anomalous values for testing purposes.

Both clients share a common base class (`BaseClient`) which encapsulates the connection, authentication and message sending logic. Subclasses only need to implement the `collect()` method.

2.2.3 Monitor

The monitor is a subscriber that registers with the `EventBus` at server startup. It receives anomaly events and logs structured alerts. It has no direct dependency on the server or on any pipeline component, it only knows about the `EventBus` interface. This makes it trivially replaceable or extensible without modifying any other part of the system.

2.2.4 Shared

The `shared` package contains three modules used across all components: `config.py` centralises all configurable parameters, `protocol.py` defines the message format and serialisation functions, and `logger.py` provides a consistent logging setup.

2.3 Concurrency Model

Sentinel uses cooperative multitasking via `asyncio` rather than thread-based concurrency. Each client connection is handled by a coroutine that suspends execution at `await` points, typically while waiting for data from the socket, allowing the event loop to service other clients in the meantime.

This approach has three advantages over thread-based alternatives: lower memory overhead, no shared-state race conditions, and predictable scheduling. The tradeoff is that all processing must be non-blocking, any CPU-bound operation would stall the event loop.

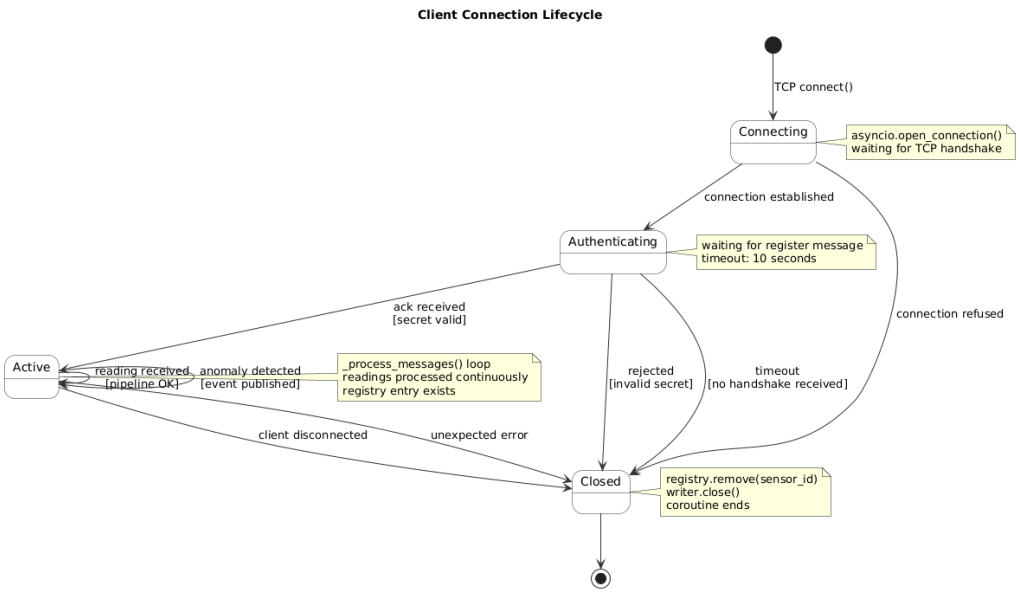

Figure 2: Client connection lifecycle

2.4 Design Decisions

2.4.1 Why asyncio over threads?

Sensor clients spend most of their time waiting between readings. Thread-based servers would allocate a full OS thread per client which expensive in memory and context-switching overhead. `asyncio` handles the same workload with a single thread and negligible overhead per coroutine.

2.4.2 Why separate PHYSICAL_LIMITS from ALERT_THRESHOLDS?

Physical limits define what is impossible in the real world regardless of context, like a CPU reading of 150%. Alert thresholds define what is considered abnormal in this specific deployment, like a CPU at 92% is possible but warrants attention. These two concerns change for different reasons and must be configured independently.

2.4.3 Why hash the shared secret?

Even in a non-production context, storing plain text secrets in source code is poor practice. The server stores a SHA-256 hash of the secret and compares hashes at authentication time. The plain text secret never reaches the server's comparison logic.

3 Communication Protocol

3.1 Overview

Sentinel uses a custom lightweight protocol built on top of TCP. Each message is a JSON object serialised to a UTF-8 string and terminated with a newline character (`\n`). The newline acts as a message delimiter, since TCP is a byte stream with no inherent message boundaries, the receiver reads until it finds a `\n` and then parses the complete JSON object.

This design keeps the protocol human-readable, language-agnostic, and easy to debug. Any TCP client capable of sending JSON can interact with the server.

3.2 Message Types

Four message types are defined in `shared/protocol.py`:

Type	Direction	Purpose
`register`	Client → Server	Authentication handshake
`reading`	Client → Server	Sensor data
`ack`	Server → Client	Registration accepted
`rejected`	Server → Client	Registration denied

Table 2: Protocol message types

3.3 Message Schemas

3.3.1 register

Sent by the client as the first message upon connection. The server will reject any message of a different type arriving before a successful registration.

```
{
    "type":       "register",
    "sensor_id": "cpu_01",
    "secret":     "sentinel-access-2026"
}
```

Listing 1: register message

3.3.2 reading

Sent periodically by the client after successful authentication.

```
{
    "type":         "reading",
    "sensor_id":    "cpu_01",
    "sensor_type": "cpu",
    "value":        78.3,
    "timestamp":    1710000000.0
}
```

Listing 2: reading message

3.3.3 ack

Sent by the server when authentication succeeds. The client may begin sending readings immediately after receiving this message.

```
{
    "type":       "ack",
    "sensor_id": "cpu_01"
}
```

Listing 3: ack message

3.3.4 rejected

Sent by the server when authentication fails. The connection is closed immediately after this message is sent.

```
{
    "type":   "rejected",
    "reason": "invalid secret"
}
```

Listing 4: rejected message

3.4 Authentication Flow

Upon connection, the client must send a `register` message within 10 seconds. The server hashes the received secret with SHA-256 and compares it against the stored hash. If the hashes match, the sensor is added to the in-memory registry and an `ack` is returned. If not, a `rejected` message is sent and the connection is closed.

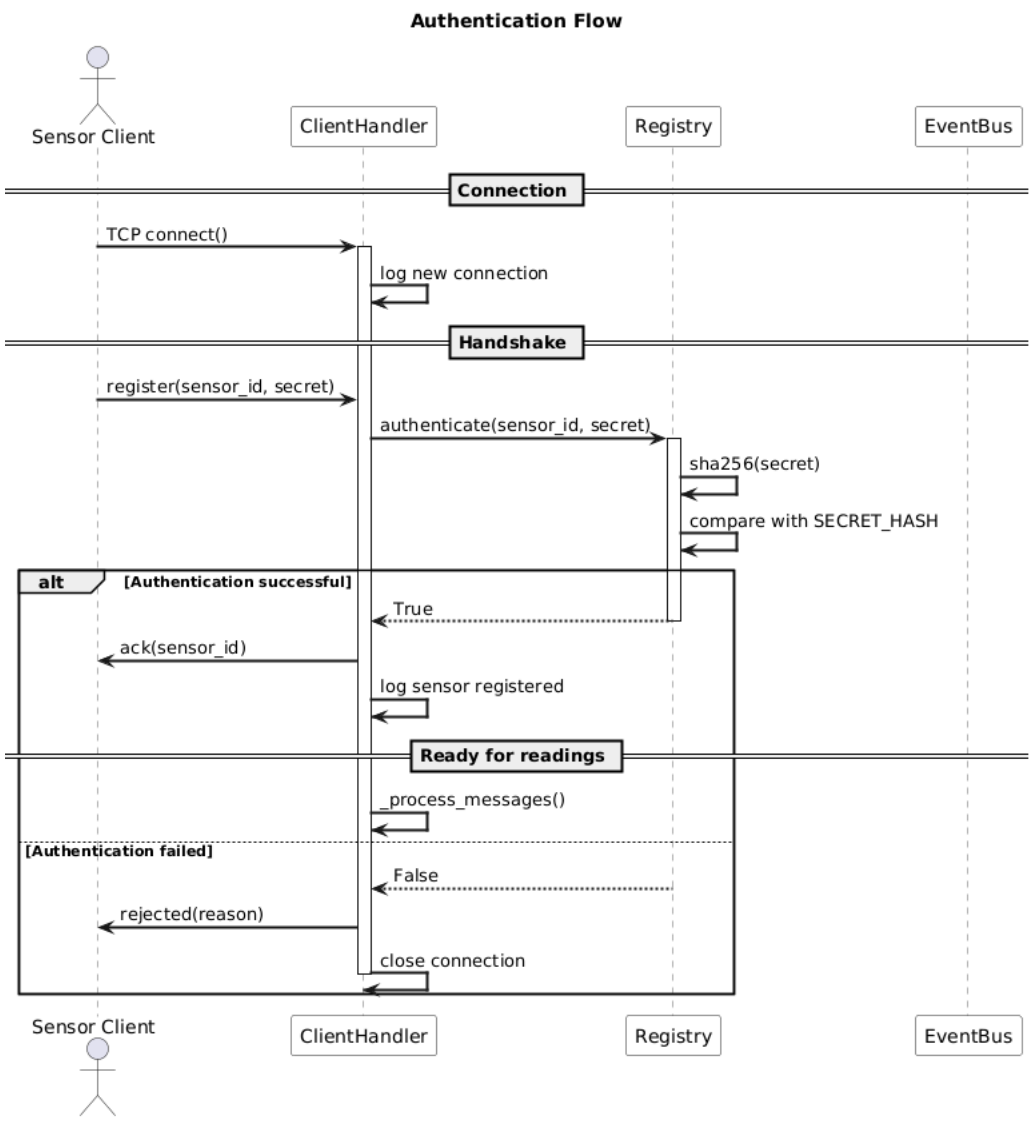

Figure 3: Authentication sequence

3.5 Reading Flow

After authentication, the client enters a loop sending `reading` messages at a configurable interval. Each message is processed by the server pipeline synchronously within the client's coroutine. The client is not notified of anomaly detections, notifications are dispatched internally via the `EventBus`.

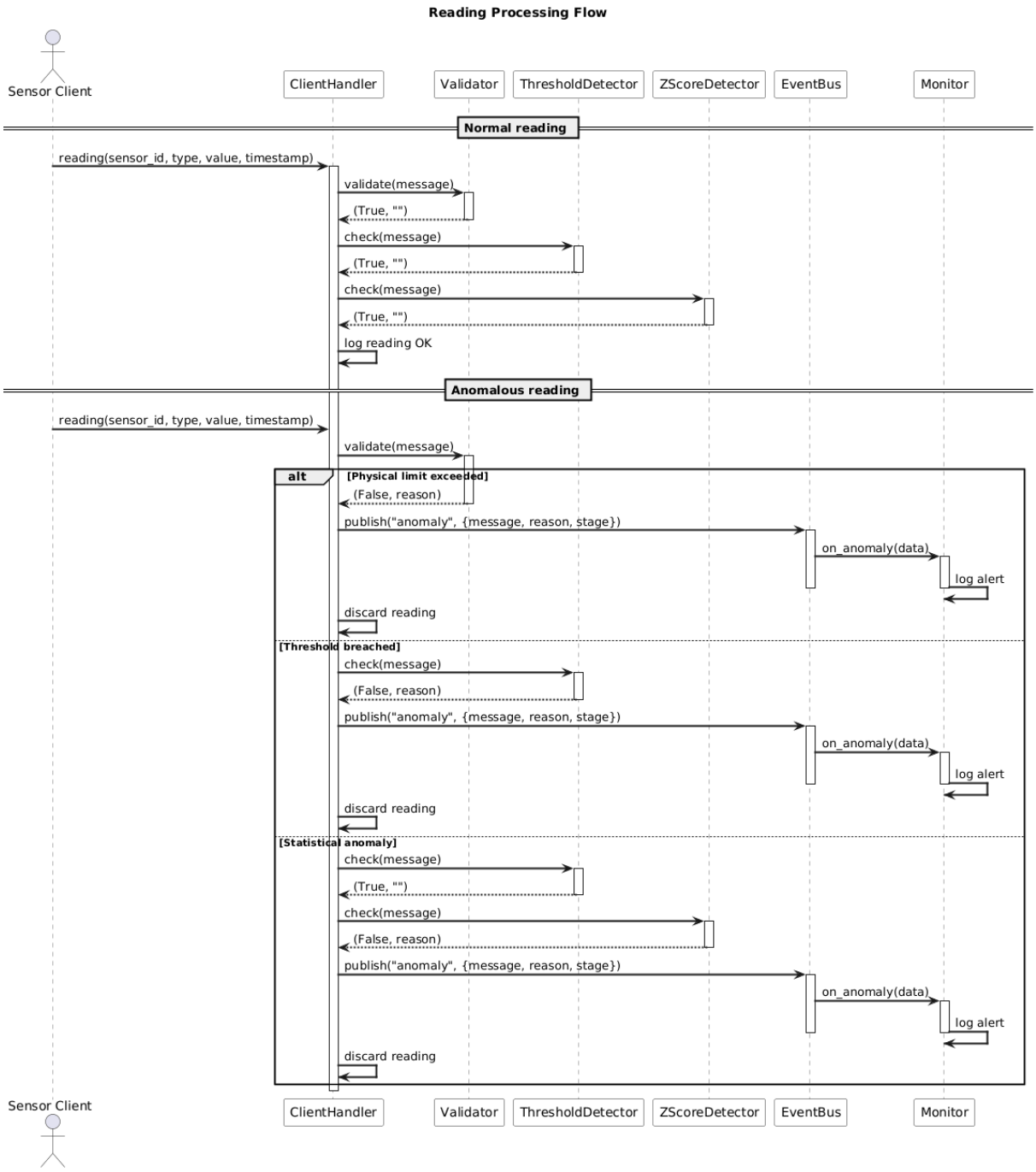

Figure 4: Reading processing sequence

4 Processing Pipeline

4.1 Overview

Every incoming `reading` message passes through a sequential three-stage pipeline before being accepted. Each stage has a single responsibility and a distinct failure mode. If a message fails at any stage, it is discarded immediately and an anomaly event is published to the `EventBus`, so the message does not proceed to the next stage.

This design follows the Single Responsibility Principle: physical boundary enforcement, operational threshold monitoring, and statistical anomaly detection are three separate concerns that change for different reasons and must be independently configurable.

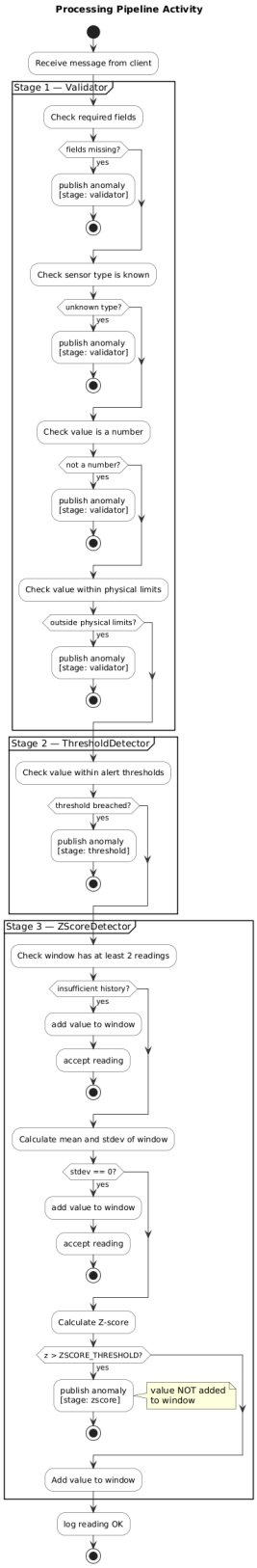

Figure 5: Processing pipeline activity diagram

4.2 Stage 1: Validator

The `Validator` is the first and most fundamental stage. It enforces two categories of rules:

Schema validation: verifies that the message contains all required fields (`sensor_id`, `sensor_type`, `value`, `timestamp`), that the sensor type is known, and that the value is a numeric type.

Physical limits: verifies that the value falls within the physically possible range for its sensor type, as defined in `PHYSICAL_LIMITS`. A CPU reading of 150% or a RAM usage of -5% are physically impossible regardless of deployment context and are rejected unconditionally.

Sensor type	Min	Max
`cpu`	0.0	100.0
`ram`	0.0	100.0
`disk`	0.0	100.0
`network`	0.0	∞

Table 3: Physical limits per sensor type

4.3 Stage 2: ThresholdDetector

The `ThresholdDetector` enforces configurable alert thresholds defined in `ALERT_THRESHOLDS`. Unlike physical limits, these thresholds are operational and define what is considered abnormal in this specific deployment, not what is physically impossible.

A CPU reading of 95% is physically valid but may warrant an alert depending on the system being monitored. Alert thresholds can be adjusted in `shared/config.py` without modifying any pipeline logic.

Sensor type	Min	Max
`cpu`	0.0	90.0
`ram`	0.0	85.0
`disk`	0.0	80.0
`network`	0.0	∞

Table 4: Default alert thresholds per sensor type

4.4 Stage 3: ZScoreDetector

The `ZScoreDetector` detects statistical anomalies using a sliding window Z-score calculation. It maintains a separate window of the last `WINDOW_SIZE` readings for each sensor, and flags any new reading that deviates more than `ZSCORE_THRESHOLD` standard deviations from the window mean.

Given a window of N readings $x_1, x_2, \ldots, x_N$ with mean μ and standard deviation σ, the Z-score of a new reading x is:

$$z = \frac{|x - \mu|}{\sigma} \tag{1}$$

If $z > K$ (where K is `ZSCORE_THRESHOLD`, default 3), the reading is flagged as a statistical anomaly.

Three edge cases are handled explicitly:

- **Insufficient history:** fewer than 2 readings in the window. The Z-score cannot be calculated. The reading is accepted and added to the window to build history.
- **Zero standard deviation:** all values in the window are identical. Division by zero is avoided by accepting the reading without Z-score evaluation.
- **Anomalous value:** the reading is flagged but not added to the window. Adding an anomalous value would shift the mean and reduce the detector's sensitivity to future anomalies.

4.5 Relationship to False Data Injection

The pipeline serves a dual purpose, it detects both sensor malfunctions and potential false data injection attempts. The three stages form a sequential defence:

1. Schema validation rejects malformed messages before any value is inspected.
2. Authentication ensures only registered sensors can submit readings.
3. Statistical detection flags values that are syntactically valid but inconsistent with the sensor's recent history, a common signature of injection attacks.

5 Design Patterns

5.1 Overview

The design patterns present in Sentinel were not chosen in advance, they emerged naturally from the architectural requirements of the system. Each pattern described in this section solves a specific design problem that arose during development, and could not be replaced by a simpler solution without introducing coupling or rigidity elsewhere.

5.2 Observer

Problem: The server needs to notify one or more components when an anomaly is detected. However, the server should not know who is listening, how many listeners there are, or what they do with the notification. Adding a direct call to the monitor from the server would couple two components that have no reason to know about each other.

Solution: The `EventBus` class implements the Observer pattern. Components that want to react to events register a callback via `subscribe()`. When the server detects an anomaly, it calls `publish()` with the event type and data and the bus dispatches the event to all registered callbacks without the server knowing who they are.

Where it appears:

```python
# Monitor registers its callback at server startup
event_bus.subscribe("anomaly", monitor.on_anomaly)

# Server publishes without knowing who is listening
event_bus.publish("anomaly", {**message, "reason": reason, "stage
    ": "validator"})
```

Listing 5: Observer - subscription and publication

Benefit: New subscribers could be added without modifying the server or any pipeline component.

5.3 Strategy

Problem: The pipeline needs to support multiple anomaly detection algorithms that can be applied independently and in sequence. Hardcoding the detection logic inside the pipeline would make it impossible to add, remove or reorder detectors without rewriting the pipeline.

Solution: Each detector (`ThresholdDetector` and `ZScoreDetector`) is an independent component with a consistent interface: a single `check(message)` method that returns a `(bool, str)` tuple. The pipeline treats them interchangeably. New detectors can be introduced by implementing the same interface.

Where it appears:

```
# Each detector exposes the same interface
normal, reason = self._threshold.check(message)
normal, reason = self._zscore.check(message)

# A new detector would be used identically
normal, reason = self._new_detector.check(message)
```

Listing 6: Strategy - interchangeable detectors

Benefit: Detection strategies are fully interchangeable. The pipeline does not depend on any specific algorithm — only on the interface.

5.4 Template Method

Problem: All sensor clients share the same connection, authentication and message sending logic. Duplicating this logic in each client would violate the DRY principle and make maintenance harder.

Solution: `BaseClient` defines the overall algorithm in `run()` (connect, authenticate, loop sending readings) and declares `collect()` as an abstract method that subclasses must implement. `PsutilClient` and `SimulationClient` only override `collect()`, leaving the rest of the algorithm untouched.

Where it appears:

```
# BaseClient defines the algorithm
async def run(self) -> None:
    connected = await self.connect()
    if not connected:
        return
    while True:
        value = self.collect()# abstract step defined by subclass
        await self.send_reading(value)
        await asyncio.sleep(self._interval)

# PsutilClient only implements the variable step
def collect(self) -> float:
    return self._collector()

# SimulationClient only implements the variable step
def collect(self) -> float:
    ...
```

Listing 7: Template Method - collect() as abstract step

Benefit: Adding a new sensor type requires implementing a single method. The connection and authentication logic is inherited automatically.

5.5 Singleton

Problem: The `Registry` and the `EventBus` must be shared across all client handlers. If each handler created its own instance, sensor registrations would not be visible to other handlers, and anomaly events would not reach the monitor.

Solution: Both instances are created once at module level in `server.py` and passed to each `ClientHandler` via constructor injection. There is exactly one `Registry` and one `EventBus` for the entire server session.

Where it appears:

```
# server.py: created once, shared by all handlers
registry = Registry()
event_bus = EventBus()
monitor = Monitor()
event_bus.subscribe("anomaly", monitor.on_anomaly)

# Each handler receives references, never creates its own
handler = ClientHandler(reader, writer, registry, event_bus)
```

Listing 8: Singleton - shared instances at module level

Benefit: All handlers operate on the same shared state without any synchronisation overhead, `asyncio`'s single-threaded model guarantees that only one coroutine accesses shared state at a time.

6 UML Diagrams

This section provides a consolidated reference for all UML diagrams produced for Sentinel. Each diagram is described with its type, purpose and the components it covers.

6.1 Class Diagram

Type: Structural, Class Diagram
Purpose: Shows the full static structure of the system, all classes, their attributes, methods and the relationships between them.
Covers: All packages (`server`, `client`, `monitor` and `shared`).

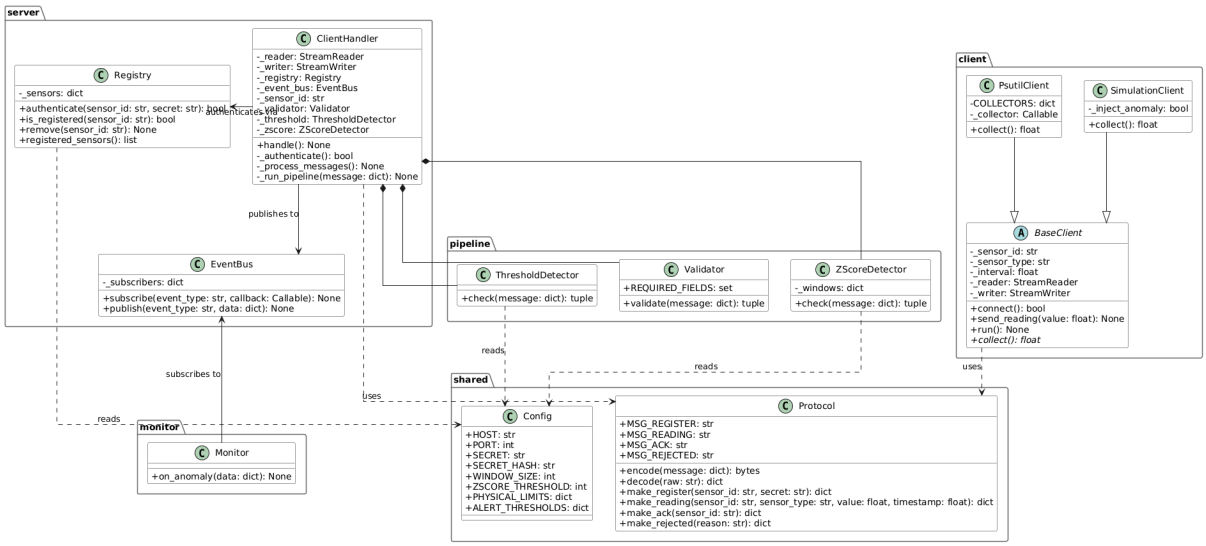

Figure 6: Class diagram

6.2 Authentication Sequence Diagram

Type: Behavioural, Sequence Diagram
Purpose: Shows the interaction between the client and the server during the authentication handshake, including both the successful and the rejected paths.
Covers: `BaseClient`, `ClientHandler`, `Registry`.

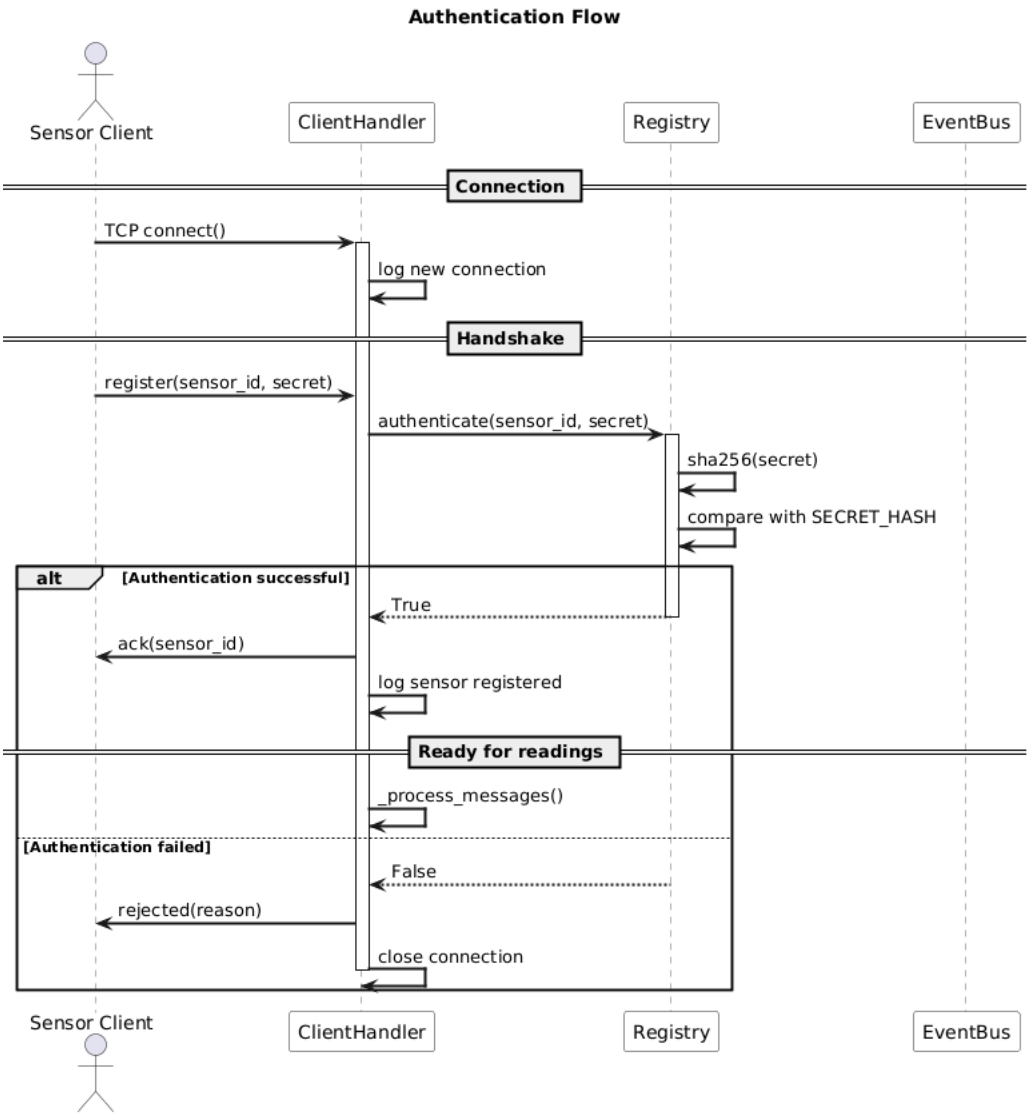

Figure 7: Authentication sequence diagram

6.3 Reading Processing Sequence Diagram

Type: Behavioural, Sequence Diagram
Purpose: Shows how a sensor reading flows through the system from the moment the client sends it to the moment the monitor receives an anomaly notification. Covers both normal and anomalous readings across all three pipeline stages.
Covers: `ClientHandler`, `Validator`, `ThresholdDetector`, `ZScoreDetector`, `EventBus`, `Monitor`.

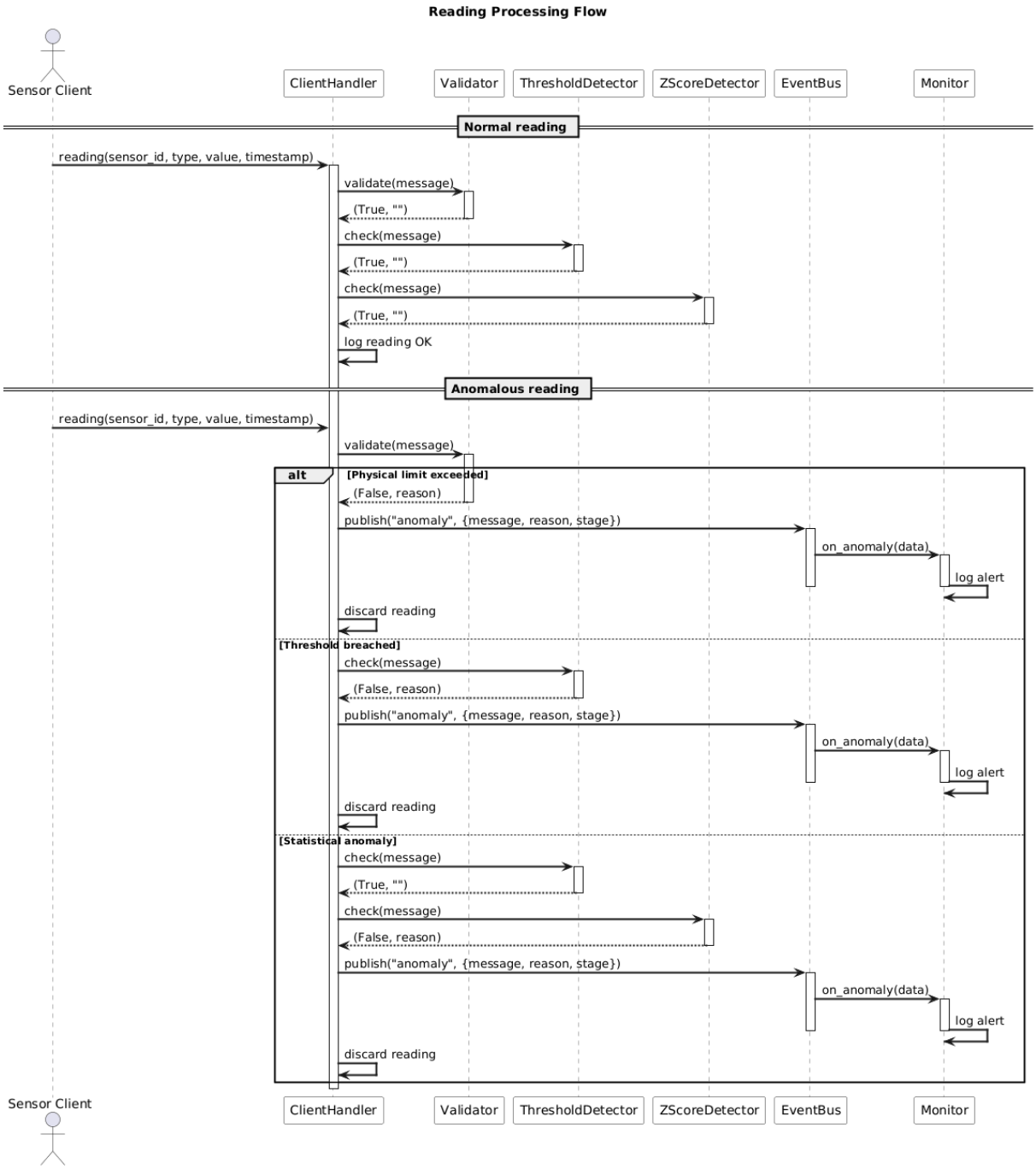

Figure 8: Reading processing sequence diagram

6.4 Processing Pipeline Activity Diagram

Type: Behavioural, Activity Diagram
Purpose: Shows the internal logic of the processing pipeline as a flow of decisions and actions. Documents every decision point, every possible exit path, and the edge cases handled by the Z-score detector.
Covers: `Validator`, `ThresholdDetector`, `ZScoreDetector`.

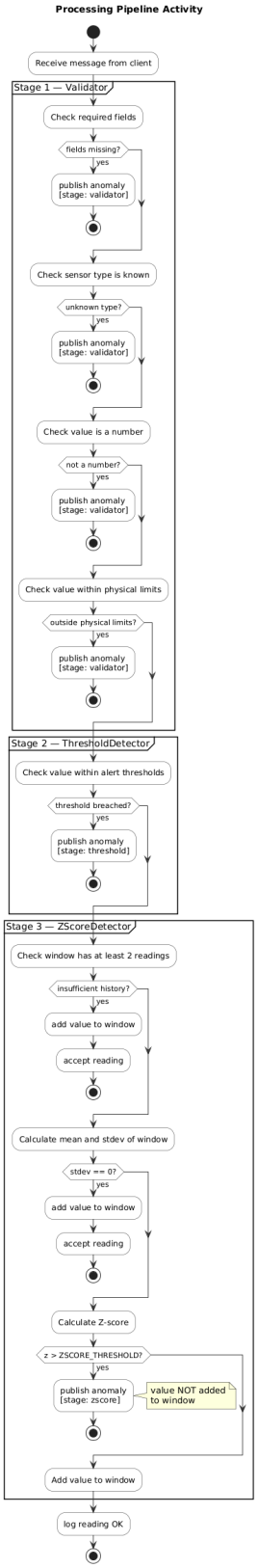

Figure 9: Processing pipeline activity diagram

6.5 Client Connection Lifecycle

Type: Behavioural, State Diagram
Purpose: Shows the states a client connection can be in and all possible transitions between them, from the initial TCP connection to the final cleanup on disconnection.
Covers: `ClientHandler`, `Registry`.

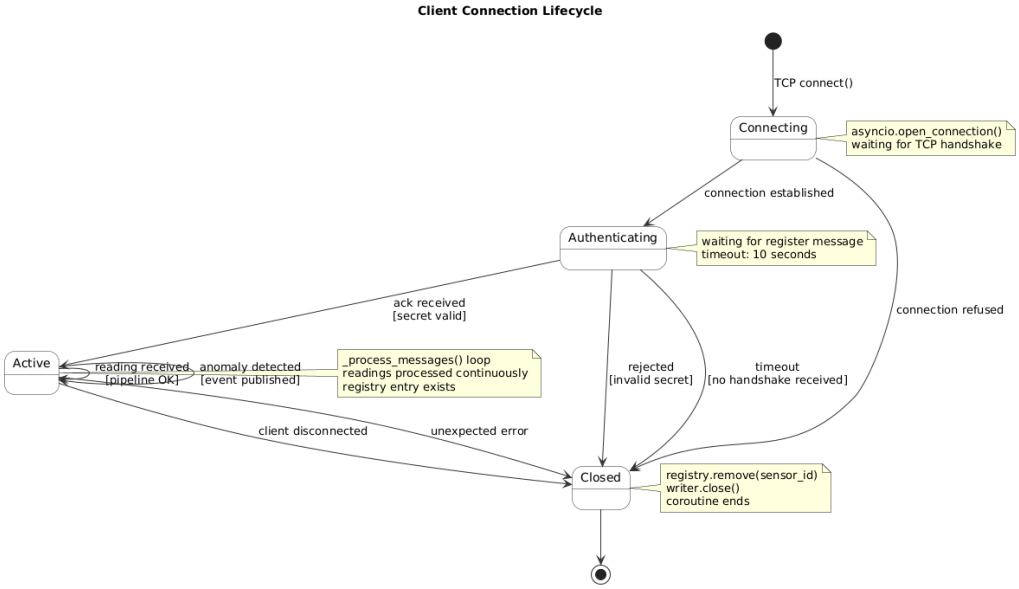

Figure 10: Client connection lifecycle state diagram

7 Configuration Reference

All configurable parameters are centralised in `shared/config.py`. Modifying this file is the only change required to adjust the behaviour of any component in the system.

7.1 Network

Parameter	Default	Description
`HOST`	`127.0.0.1`	IP address the server listens on.
`PORT`	`9999`	TCP port the server listens on.

Table 5: Network parameters

7.2 Authentication

Parameter	Default	Description
`SECRET`	`sentinel-access-2026`	Plain text shared secret known only to legitimate clients. Never stored or compared directly on the server side.
`SECRET_HASH`	*derived*	SHA-256 hash of `SECRET`. Computed automatically at startup. This is the value the server compares against.

Table 6: Authentication parameters

7.3 Pipeline

Parameter	Default	Description
`WINDOW_SIZE`	`10`	Number of readings maintained in the sliding window per sensor for Z-score calculation. A larger window produces more stable statistics but reacts more slowly to trend changes.
`ZSCORE_THRESHOLD`	`3`	Number of standard deviations above which a reading is flagged as a statistical anomaly. A value of 3 corresponds to the three-sigma rule, which flags approximately 0.3% of readings in a normal distribution.

Table 7: Pipeline parameters

7.4 Physical Limits

Physical limits define the boundaries of what is physically possible for each sensor type. Values outside these ranges are rejected by the `Validator` regardless of any other configuration.

Sensor type	Min	Max
`cpu`	0.0	100.0
`ram`	0.0	100.0
`disk`	0.0	100.0
`network`	0.0	∞

Table 8: Physical limits per sensor type

7.5 Alert Thresholds

Alert thresholds define the operational boundaries for each sensor type. Values outside these ranges are flagged by the `ThresholdDetector` as anomalous. These values are independent of physical limits and are intended to be adjusted to match the specific deployment environment.

Sensor type	Min	Max
`cpu`	0.0	90.0
`ram`	0.0	85.0
`disk`	0.0	80.0
`network`	0.0	∞

Table 9: Alert thresholds per sensor type

8 Setup and Usage

8.1 Requirements

- Python 3.10 or later
- pip
- A terminal with support for multiple simultaneous sessions

8.2 Installation

Clone the repository and create a virtual environment:

```
git clone https://github.com/pedromedinatech/sentinel.git
cd sentinel
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

Listing 9: Installation

8.3 Running the System

Each component runs in a separate terminal. The server must be started before any client attempts to connect.

8.3.1 Start the server

```
python -m server.server
```

Listing 10: Starting the server

Expected output:

```
2026-04-14 10:00:00  INFO  __main__  sentinel server listening on
    127.0.0.1:9999
```

Listing 11: Server startup output

8.3.2 Connect a real sensor client

Collects live system metrics from the machine it runs on using `psutil`:

```
python -m client.psutil_client
```

Listing 12: Starting the psutil client

8.3.3 Connect a simulated sensor client

Generates readings within the configured alert threshold range:

```
python -m client.simulation_client <sensor_id> <sensor_type>

# Examples
python -m client.simulation_client cpu_sim cpu
python -m client.simulation_client ram_sim ram
python -m client.simulation_client disk_sim disk
```

Listing 13: Starting a simulated client

8.3.4 Inject anomalous values

Generates readings that deliberately exceed the alert thresholds, triggering the detection pipeline:

```
python -m client.simulation_client cpu_sim cpu inject
```

Listing 14: Starting an injection client

8.3.5 Available sensor types

Type	psutil source	Unit
cpu	psutil.cpu_percent()	%
ram	psutil.virtual_memory().percent	%
disk	psutil.disk_usage("/").percent	%
network	psutil.net_io_counters().bytes_sent	bytes

Table 10: Available sensor types and their data sources

8.4 Demo Scenario

The following sequence demonstrates the full system in operation. Open four terminals, activate the virtual environment in each one, and run the commands in order:

Terminal	Command	Purpose
1	python -m server.server	Start the server
2	python -m client.psutil_client	Real system metrics
3	python -m client.simulation_client ram_sim ram	Simulated normal readings
4	python -m client.simulation_client cpu_sim cpu inject	Anomaly injection

Table 11: Demo scenario terminal setup

Terminal 1 will show all incoming connections, pipeline results and anomaly alerts in real time. Anomalies injected from terminal 4 will appear immediately in the server log with the stage that detected them and the reason for the alert.

8.5 Stopping the System

Press `Ctrl+C` in any terminal to stop that component. The server logs a clean shutdown message and releases the socket. Clients detect the disconnection and close their connections automatically.

9 Conclusion

Sentinel demonstrates that concepts such as cooperative concurrency, event-driven architecture and layered processing pipelines are not abstract theory but concrete engineering decisions that solve real problems. Each architectural choice in the system has a clear motivation: `asyncio` was chosen because thread overhead is unjustified for I/O-bound workloads; the pipeline was split into three stages because physical limits, operational thresholds and statistical behaviour change for different reasons; the `EventBus` was introduced because the server has no reason to know who reacts to anomalies.

The result is a system where each component has a single, well-defined responsibility, components are loosely coupled through explicit interfaces, and the design patterns present in the codebase emerged from necessity rather than being imposed from the outside.

9.1 Current Limitations

The current version of Sentinel has a number of known limitations that are acceptable in the context of this project but would need to be addressed before any production use:

- The shared secret is stored in plain text in the source code. In a production system it would be loaded from an environment variable or a secrets manager.
- There is no persistence. Anomaly history, sensor registrations and configuration changes are lost when the server stops. A database or a structured log file would be required for any operational use.
- The monitor only logs alerts to stdout. It has no way to notify external systems or persist the anomaly record.
- The system has no reconnection logic. If the server goes down, clients terminate and must be restarted manually.
- Authentication uses a single shared secret for all sensors. A more robust system would issue individual tokens per sensor, allowing individual sensors to be revoked without affecting the rest.

9.2 Possible Extensions

The architecture of Sentinel is designed to accommodate growth without requiring structural changes. The following extensions are natural next steps:

- **Persistence layer:** anomaly events published to the `EventBus` could be consumed by a subscriber that writes them to a database or a structured log file, without any modification to the server or the pipeline.
- **REST API:** a subscriber could expose an HTTP endpoint serving the anomaly history and current sensor status, enabling integration with external dashboards or monitoring tools.
- **Live dashboard:** a web interface consuming the REST API could display real-time sensor readings and anomaly alerts visually.

- **External monitor clients:** the monitor could be moved out of the server process and connect as a TCP client, subscribing to a dedicated notification channel. This would allow multiple independent monitors to run on different machines.
- **Per-sensor authentication tokens:** replacing the shared secret with individual tokens would allow fine-grained access control and sensor-level revocation.
- **Additional sensor types:** new sensor types can be added by extending `PHYSICAL_LIMITS` and `ALERT_THRESHOLDS` in `config.py` and adding the corresponding collector to `PsutilClient.COLLECTORS`.

9.3 Final Remarks

The development of Sentinel covered a broad range of software engineering concepts in a single coherent project: network programming, asynchronous concurrency, object-oriented design, design patterns, statistical methods and formal documentation.